



50% Individual Coursework

2025-26 Autumn

Credit: 30

Student Name: Prasidha Kumar Sarawagi

London Met ID: PKS0076

CollegeID: np01ai4a250040

Assignment Due Date: 17/04/2026

Assignment Submission Date: 17/04/2026

Module Leader: Mr. Bikram Poudel

Word Count:

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

1. Introduction
2. Milestone 2 Requirement Summary
3. Class Diagram
4. Pseudocode
5. Method Descriptions
6. GUI Description
7. Testing
8. Error Detection and Correction
9. Conclusion and Reflection
10. References
11. Appendix

1. Introduction

This report documents Milestone 2 of the CS4001NI Programming coursework. The coursework is based on an AI model subscription manager written in Java. The application stores AI model subscription plans and allows the user to manage personal and pro plan actions through a graphical user interface.

Milestone 2 focuses on the Swing GUI, ArrayList storage, polymorphism, file handling, abstraction, and token-based plan logic. The updated code uses an abstract AIModel class, PersonalPlan for users with available tokens, and ProPlan for team users with team slots.

2. Milestone 2 Requirement Summary

Milestone 2 required the coursework to move beyond the basic class structure and include a working GUI controller. The GUI must allow the user to add plans, display plans, validate index input, check plan type, handle prompts, manage team members, export data, and load data from a file.

The lecturer also clarified updated logic for token handling. These updates are reflected in the report and in the Java code used for this submission.

- AIModel is an abstract class.
- display() in AIModel is abstract and is implemented by the child classes.
- contextWindow is stored as an integer numeric value.
- AIModel includes calculateTotalToken(String promptText, int expectedOutputTokens).
- PersonalPlan uses availableTokens instead of promptsRemaining.
- PersonalPlan has purchaseTokens(int tokens), which adds purchased tokens to availableTokens.
- PersonalPlan enterPrompt() calculates total token usage and deducts tokens when enough tokens are available.
- ProPlan enterPrompt() calculates token usage but does not reduce a token balance.
- SubscriptionGUI uses ArrayList<AIModel>, instanceof checks, try/catch index validation, JOptionPane messages, export, and load.

3. Class Diagram

The following text-based class diagram shows the inheritance relationship and the main members from the submitted code. AIModel is the parent class. PersonalPlan and ProPlan extend AIModel. SubscriptionGUI uses an ArrayList of AIModel objects.

```
<<abstract>> AIModel
-----
- modelName: String
- cost: double
- parameters: int
- contextWindow: int
-----
+ AIModel(String modelName, double cost, int parameters, int contextWindow)
+ getModelName(): String
+ getCost(): double
+ getParameters(): int
+ getContextWindow(): int
+ calculateTotalToken(String promptText, int expectedOutputTokens): int
+ display(): String <<abstract>>

AIModel -> PersonalPlan
-----
- availableTokens: int
-----
+ PersonalPlan(String modelName, double price, int parameters, int contextWindow, int
availableTokens)
+ getAvailableTokens(): int
```

```
+ purchaseTokens(int tokens): String
+ enterPrompt(String promptText, int expectedOutputTokens): String
+ display(): String
```

```
AIModel -> ProPlan
```

```
-----
- teamSlots: int
-----
```

```
+ ProPlan(String modelName, double price, int parameters, int contextWindow, int teamSlots)
+ getTeamSlots(): int
+ addTeamMember(String memberName): String
+ removeTeamMember(String memberName): String
+ enterPrompt(String promptText, int expectedOutputTokens): String
+ display(): String
```

```
SubscriptionGUI extends JFrame and implements ActionListener
```

```
-----
- planList: ArrayList<AIModel>
- text fields for plan details, prompt details, member name, and index
- buttons for adding plans, displaying, clearing, prompting, team actions, type checking, export, and load
-----
+ SubscriptionGUI()
+ actionPerformed(ActionEvent e): void
- getIndex(): int
- exportToFile(): void
- loadFromFile(): void
+ main(String[] args): void
```

4. Pseudocode

4.1 Add Personal Plan Button

```
WHEN Add Personal Plan button is clicked
    READ model name, price, parameters, context window and token amount from text fields
    TRY to convert price to double
    TRY to convert parameters, context window and token amount to integers
    IF model name is empty THEN
        show input error message
    ELSE
        create new PersonalPlan object
        add object to ArrayList<AIModel>
        print success message with index number
    END IF
CATCH any invalid input
    show JOptionPane input error message
END WHEN
```

4.2 Add Pro Plan Button

```
WHEN Add Pro Plan button is clicked
    READ model name, price, parameters, context window and team slots from text fields
    TRY to convert numeric fields
    IF model name is empty THEN
        show input error message
    ELSE
        create new ProPlan object
        add object to ArrayList<AIModel>
        print success message with index number
    END IF
CATCH any invalid input
    show JOptionPane input error message
END WHEN
```

4.3 Give Prompt Button

```

WHEN Give Prompt button is clicked
  CALL getIndex()
  IF index is -1 THEN stop
  READ prompt text and output token value
  GET AIModel object from planList using the index
  IF object is PersonalPlan THEN
    cast object to PersonalPlan
    call enterPrompt(promptText, outputTokens)
  ELSE IF object is ProPlan THEN
    cast object to ProPlan
    call enterPrompt(promptText, outputTokens)
  ELSE
    print unknown plan type
  END IF
CATCH invalid output token number
  show JOptionPane input error message
END WHEN

```

4.4 Get Index Method

```

FUNCTION getIndex()
  SET idx to -1
  TRY to convert index text field to integer
    IF idx is less than 0 OR idx is greater than or equal to planList size THEN
      show JOptionPane index error message
      return -1
    END IF
  CATCH NumberFormatException
    show JOptionPane input error message
    return -1
  END TRY
  return idx
END FUNCTION

```

4.5 Check Plan Type

```

WHEN Check Plan Type button is clicked
  CALL getIndex()
  IF index is -1 THEN stop
  GET AIModel object from planList
  IF object is PersonalPlan THEN
    print Personal Plan message
  ELSE IF object is ProPlan THEN
    print Pro Plan message
  ELSE
    print unknown plan type
  END IF
END WHEN

```

4.6 Export to File

```

WHEN Export button is clicked
  OPEN plans_export.txt using PrintWriter and FileWriter
  FOR each plan in planList
    write plan index
    write plan.display() output
    write a blank line
  END FOR
  close file
  print export success message
CATCH IOException
  print export failed message
END WHEN

```

4.7 Load From File

```

WHEN Load button is clicked
  OPEN plans_export.txt using BufferedReader and FileReader
  CLEAR output area
  WHILE file has another line
    append line to output area
  END WHILE
  close file
  print file loaded message
CATCH IOException
  print load failed message
END WHEN

```

5. Method Descriptions

5.1 AIModel.java

Method name	Parameters	Return type	Brief description	Internal working
AIModel	String modelName, double cost, int parameters, int contextWindow	Constructor	Creates the parent AI model object.	Stores the model name, cost, parameter count and numeric context window in private attributes.
getModelName	none	String	Returns the model name.	Returns the value stored in modelName.
getCost	none	double	Returns the cost.	Returns the value stored in cost.
getParameters	none	int	Returns the parameter count.	Returns the value stored in parameters.
getContextWindow	none	int	Returns the context window.	Returns the numeric contextWindow value.
calculateTotalToken	String promptText, int expectedOutputTokens	int	Calculates total tokens for a prompt.	Counts input tokens by splitting promptText into words, adds expectedOutputTokens, and throws IllegalArgumentException if totalTokens is greater than contextWindow.
display	none	String	Abstract display method.	No body is provided in AIModel. Child classes must override this method.

5.2 PersonalPlan.java

Method name	Parameters	Return type	Brief description	Internal working
PersonalPlan	String modelName, double price, int parameters, int contextWindow, int availableTokens	Constructor	Creates a personal plan.	Calls super() to initialise AIModel fields and stores availableTokens.
getAvailableTokens	none	int	Returns available tokens.	Returns the current value of availableTokens.
purchaseTokens	int tokens	String	Purchases extra tokens.	Checks if tokens is negative. If negative, returns an error message. Otherwise adds tokens to availableTokens and returns a success message.
enterPrompt	String promptText, int expectedOutputTokens	String	Processes a prompt for PersonalPlan.	Calls calculateTotalToken(). If enough tokens are available, deducts totalTokens from availableTokens and returns prompt details. If not enough tokens are available, returns an error message.
display	none	String	Displays personal plan details.	Returns model details from getter methods and adds availableTokens.

5.3 ProPlan.java

Method name	Parameters	Return type	Brief description	Internal working
ProPlan	String modelName, double price, int parameters, int contextWindow, int teamSlots	Constructor	Creates a pro plan.	Calls super() to initialise AIModel fields and stores teamSlots.
getTeamSlots	none	int	Returns team slots.	Returns the current value of teamSlots.
addTeamMember	String memberName	String	Adds a team member.	If teamSlots is greater than zero, it reduces teamSlots by one and returns a success message. Otherwise it returns an error message.
removeTeamMember	String memberName	String	Removes a team member.	Increases teamSlots by one and returns a message showing the member removed.
enterPrompt	String promptText, int expectedOutputTokens	String	Processes a prompt for ProPlan.	Calls calculateTotalToken() and returns token usage. It does not reduce any token balance.
display	none	String	Displays pro plan details.	Returns model details from getter methods and adds teamSlots.

6. GUI Description

SubscriptionGUI is the main graphical user interface class. It extends JFrame and implements ActionListener. It creates the input fields, buttons and output area used by the application.

The GUI stores all plans in an ArrayList<AIModel> called planList. This allows both PersonalPlan and ProPlan objects to be stored in the same collection because both classes inherit from AIModel.

6.1 Text Fields

- Model Name: accepts the AI model name.
- Price: accepts the model price per 1 Lakh tokens.
- Parameters: accepts the model parameter count in billions.
- Context Window: accepts a numeric context window such as 96000.
- Tokens / Slots: used as available tokens for PersonalPlan or team slots for ProPlan.
- Prompt Text: accepts the prompt entered by the user.
- Output Tokens: accepts expected output tokens.
- Member Name: accepts the name used for team member actions.
- Index Number: selects a plan from the ArrayList.

6.2 Buttons and Actions

- Add Personal Plan creates a PersonalPlan object and adds it to planList.
- Add Pro Plan creates a ProPlan object and adds it to planList.
- Display All Plans loops through planList and prints each plan display output.
- Clear Fields clears all text fields.
- Give a Prompt calls enterPrompt() for either PersonalPlan or ProPlan depending on the object type.
- Add Team Member and Remove Team Member work only for ProPlan objects.
- Check Plan Type uses instanceof to show whether the selected object is PersonalPlan or ProPlan.
- Export to File writes plan details to plans_export.txt.
- Load from File reads plans_export.txt and displays the contents in the output area.

6.3 Validation and Error Handling

The getIndex() method uses try/catch to check that the index is an integer and that it is within the valid ArrayList range. If the index is invalid, a JOptionPane error message is shown and the method returns -1. Other input errors, such as invalid numeric values, are also shown using JOptionPane message dialogs.

7. Testing

This section provides test cases for Milestone 2. The screenshots below are original clean screenshots taken from the completed program evidence.

Test 1: Compile and run program

Item	Details
Objective	To check that all Java files compile and the program starts.
Expected result	The program should compile without errors and the GUI should open.
Actual result	Program compiled successfully in BlueJ, and the SubscriptionGUI window opened successfully.
Short explanation	This test confirms successful compilation and GUI launch.



Figure 1. BlueJ project showing the compiled coursework classes.



Figure 2. SubscriptionGUI running successfully.

Test 2: Add Personal Plan and Pro Plan

Item	Details
Objective	To check that both plan types can be added to the ArrayList.
Expected result	The output area should show that Personal Plan and Pro Plan were added with index numbers.
Actual result	Personal and Pro plans were added successfully, and Display All showed the saved plan details.
Short explanation	This test confirms plan creation, ArrayList storage and display output.

The screenshot displays the 'AI Model Subscription Manager' application interface. The title bar indicates 'CS4001 Java OOP Coursework'. The main window is divided into several sections:

- Plan Details:** A form with fields for 'Model Name' (chatgpt personal), 'Price (Rs/IL tok)' (250), 'Parameters (B)' (70), 'Context Window' (8000), 'Tokens / Slots' (5000), 'Personal / Pro Prompt', 'Prompt Text', 'Output Tokens', 'Pro Plan Actions', 'Member Name', 'Index Lookup', and 'Index Number'.
- Actions:** A sidebar containing buttons for 'ADD PLANS' (Add Personal Plan, Add Pro Plan), 'VIEW & MANAGE' (Display All Plans, Check Plan Type, Clear Fields), 'PROMPT' (Give a Prompt), 'PRO PLAN' (Add Team Member, Remove Team Member), and 'FILE I/O' (Export to File, Load from File).
- Output Console:** A text area at the bottom showing the system message: 'System ready. Add a plan to begin. Personal Plan added at index [0]'.

Figure 3. Personal Plan added through the GUI.

The screenshot shows the 'AI Model Subscription Manager' application. The 'Plan Details' section on the left contains the following information:

Plan Details	
Model Name	chatgpt pro
Price (Rs/1L tok)	900
Parameters (B)	120000
Context Window	90000
Tokens / Slots	5
Personal / Pro Prompt	
Prompt Text	create an ad campaign
Output Tokens	120
Pro Plan Actions	
Member Name	Sita Sharma
Index Lookup	
Index Number	2

The 'Actions' section on the right contains the following buttons:

- ADD PLANS: Add Personal Plan, Add Pro Plan
- VIEW & MANAGE: Display All Plans, Check Plan Type, Clear Fields
- PROMPT: Give a Prompt
- PRO PLAN: Add Team Member, Remove Team Member
- FILE I/O: Export to File, Load from File

The 'Output Console' at the bottom displays the following text:

```
Prompt: explain object oriented programming concept in java
Expected output tokens: 100
Total tokens used: 107
Remaining tokens: 4893
Not enough available tokens in Personal Plan.
Pro Plan added at index [2]
Sita Sharma added, Slots remaining: 4
```

Figure 4. Pro Plan action shown through the GUI.

The screenshot shows the 'AI Model Subscription Manager' application. The 'Plan Details' section on the left contains the following information:

Plan Details	
Model Name	chatgpt pro
Price (Rs/1L tok)	900
Parameters (B)	175
Context Window	16000
Tokens / Slots	5
Personal / Pro Prompt	
Prompt Text	
Output Tokens	
Pro Plan Actions	
Member Name	
Index Lookup	
Index Number	

The 'Actions' section on the right contains the following buttons:

- ADD PLANS: Add Personal Plan, Add Pro Plan
- VIEW & MANAGE: Display All Plans, Check Plan Type, Clear Fields
- PROMPT: Give a Prompt
- PRO PLAN: Add Team Member, Remove Team Member
- FILE I/O: Export to File, Load from File

The 'Output Console' at the bottom displays the following text:

```
----- Plan Index: 0 -----
AI Model: chatgpt personal
Subscription Cost: Rs. 250.0 per 1 Lakh tokens
Model Size: 70 billion params
Context Window: 8000
Available Tokens: 5000

----- Plan Index: 1 -----
AI Model: chatgpt pro
```

Figure 5. Display All output showing the saved plan details.

Test 3: Give Prompt

Item	Details
Objective	To check prompt execution and token calculation.
Expected result	PersonalPlan should reduce available tokens. ProPlan should show token usage without reducing a token balance.
Actual result	The prompt was executed successfully. Total tokens were calculated and remaining tokens were reduced for the Personal Plan.
Short explanation	This test confirms calculateTotalToken() and PersonalPlan token deduction.

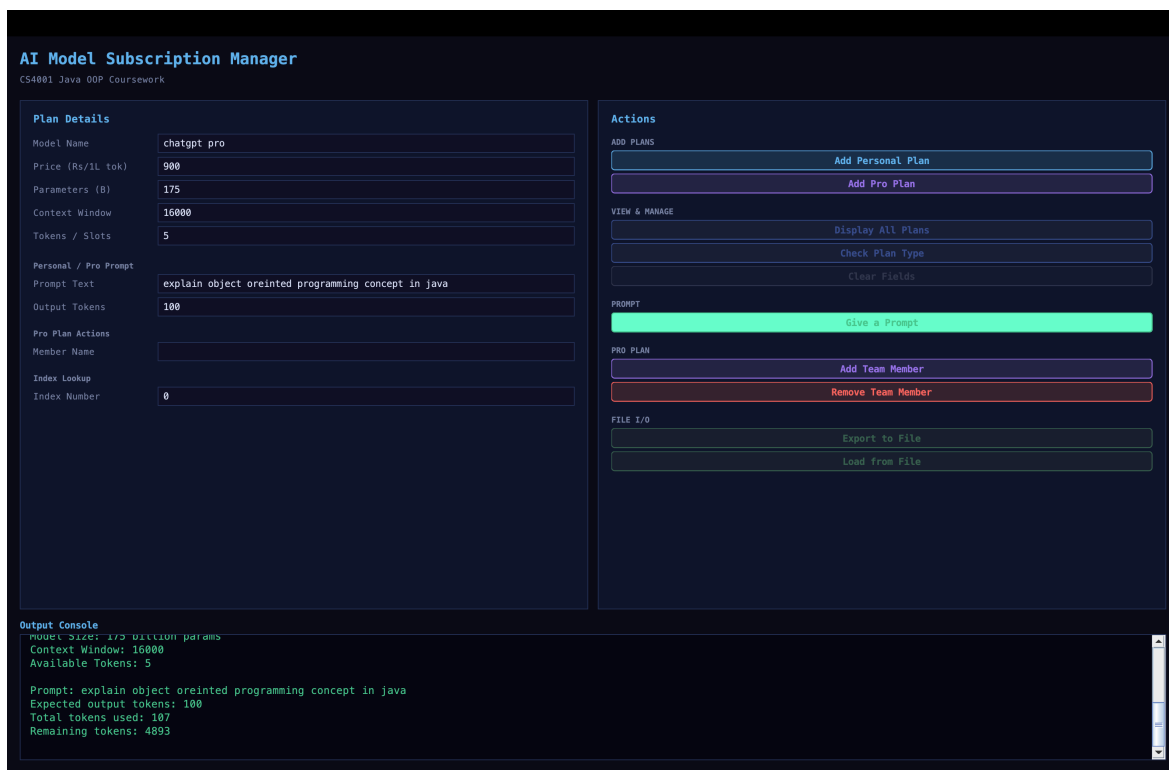


Figure 6. Prompt execution showing token calculation and deduction.

Test 4: Add Team Member

Item	Details
Objective	To check ProPlan team member management.
Expected result	The selected ProPlan should add the member and reduce teamSlots by one.
Actual result	A team member was added to the Pro Plan and the available slot count decreased.
Short explanation	This test confirms ProPlan team member and slot management.

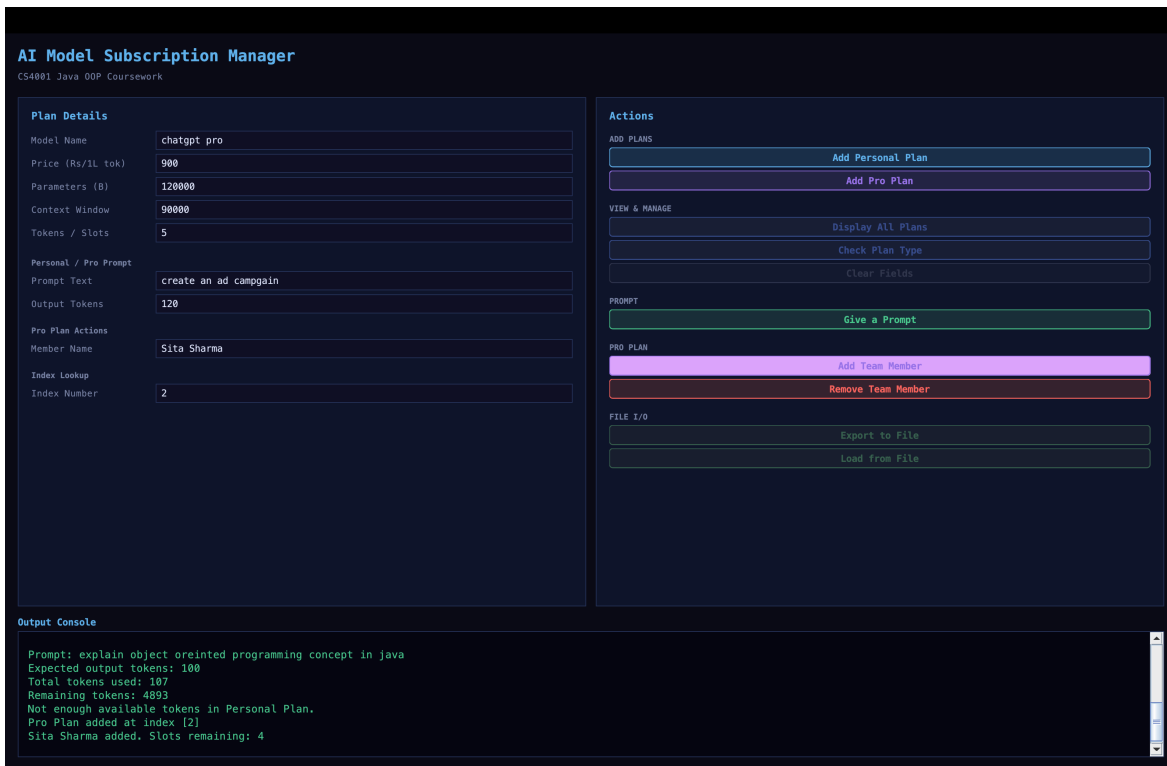


Figure 7. Team member added to the Pro Plan.

Test 5: Invalid index error message

Item	Details
Objective	To check index validation and JOptionPane error handling.
Expected result	The program should show an error message for non-integer or out-of-range index input.
Actual result	An invalid index value displayed a JOptionPane error message.
Short explanation	This test confirms getIndex() validation and dialog-based error handling.

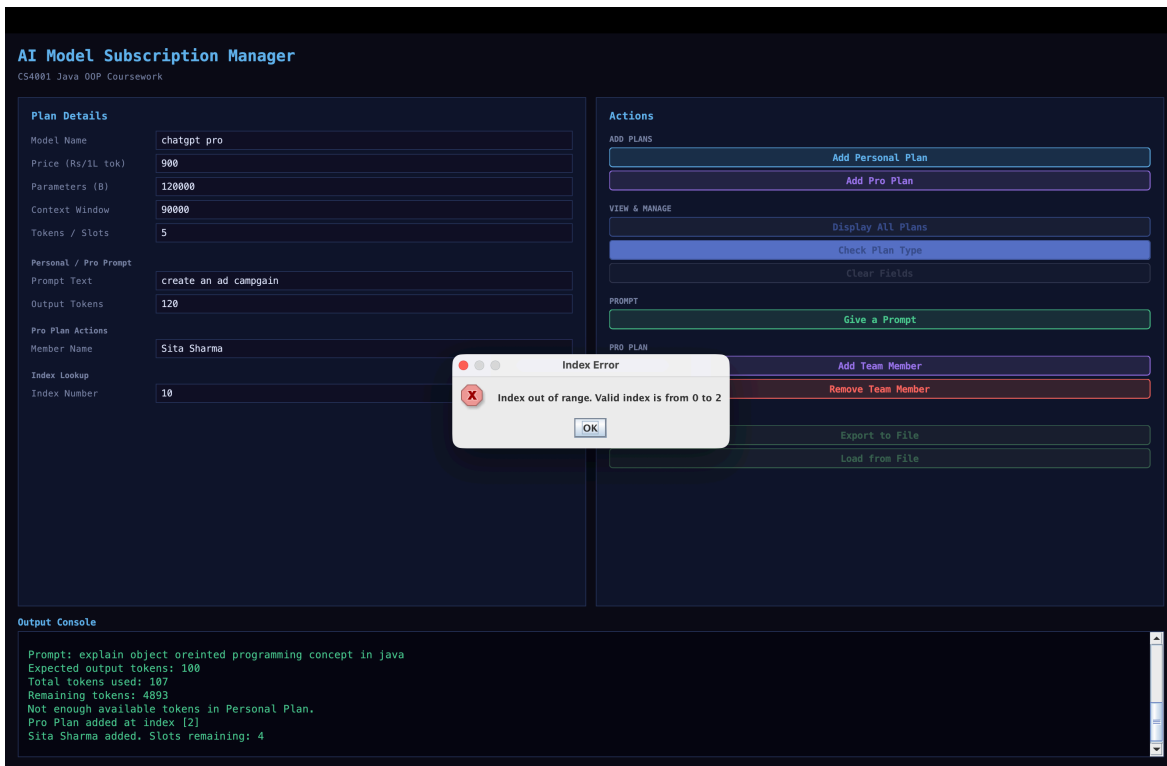


Figure 8. Error dialog displayed for an invalid index value.

Test 6: Export to file and load from file

Item	Details
Objective	To check file writing and file loading.
Expected result	The program should export plan data to plans_export.txt and load it back into the output area.
Actual result	The plan data was exported to plans_export.txt and loaded back into the GUI output area successfully.
Short explanation	This test confirms export to file and load from file behavior.

```

plans_export.txt -- Edited
AI Model: chatgpt personal
Subscription Cost: Rs. 250.0 per 1 Lakh tokens
Model Size: 70 billion params
Context Window: 8000
Available Tokens: 4893

=== Plan [1] ===
AI Model: chatgpt pro
Subscription Cost: Rs. 900.0 per 1 Lakh tokens
Model Size: 175 billion params
Context Window: 16000
Available Tokens: 5

=== Plan [2] ===
AI Model: chatgpt pro
Subscription Cost: Rs. 900.0 per 1 Lakh tokens
Model Size: 120000 billion params
Context Window: 90000
Team Slots Available: 4

```

Figure 9. Exported file showing saved plan information.

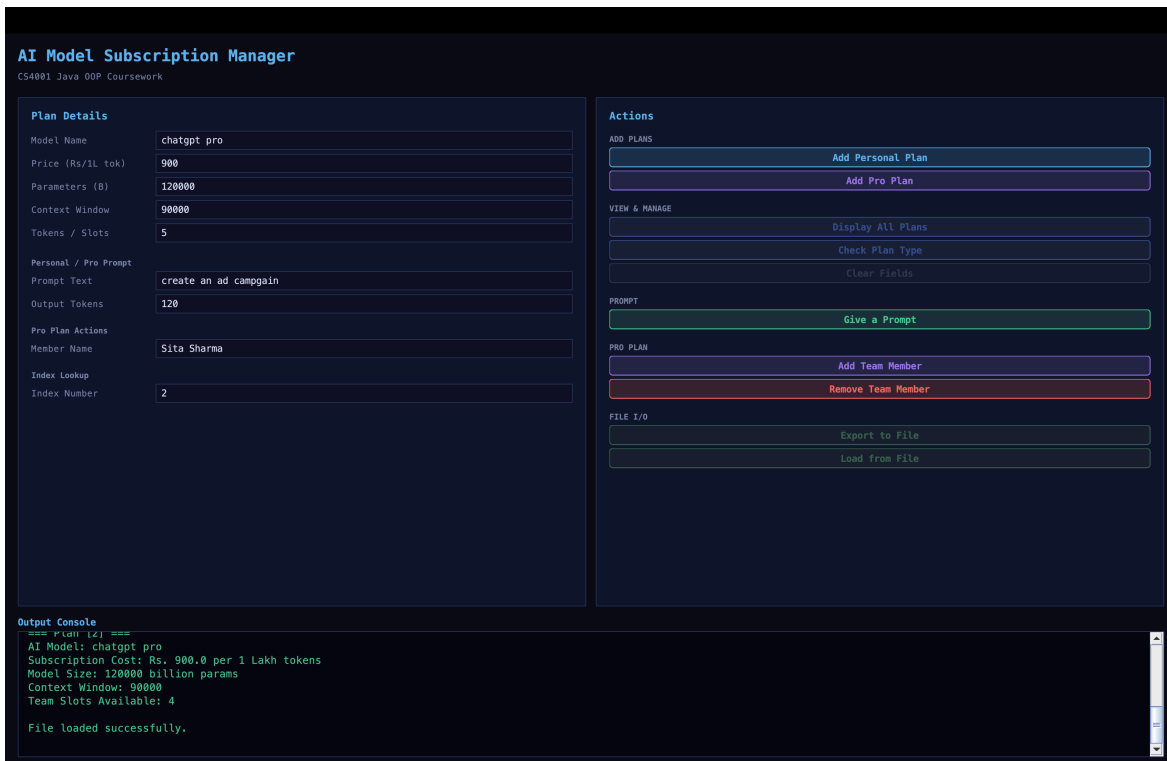


Figure 10. Loaded file content displayed successfully in the GUI.

8. Error Detection and Correction

During development and testing, common errors were checked and corrected. The examples below describe realistic error types in this project and explain how they were handled or corrected.

8.1 Syntax Error

Part	Details
Description of the error	A syntax error can occur if a bracket, semicolon or method signature is typed incorrectly while editing one of the Java classes.
How it was identified	It is identified during compilation in BlueJ or terminal because Java shows a compile-time error.
Solution	The missing or incorrect syntax is corrected, and the project is compiled again.
Evidence	Evidence can be shown using the BlueJ compile screenshot or a corrected code snippet.

8.2 Runtime/Input Error

Part	Details
Description of the error	A runtime/input error can occur if the user enters text into a numeric field such as price, parameters, context window, output tokens or index.
How it was identified	It is identified when the button is clicked and the program cannot convert the text to a number.
Solution	The GUI uses try/catch and JOptionPane messages to inform the user about invalid input.
Evidence	Evidence can be shown using the related GUI input validation message.

8.3 Logic/Input Validation Error

Part	Details
Description of the error	A logic/input validation issue can occur if an invalid index is accepted or if a ProPlan action is attempted on a PersonalPlan object.
How it was identified	It is identified during GUI testing by entering an invalid index or using a team action on the wrong plan type.
Solution	The getIndex() method checks the valid range, and instanceof checks the selected plan type before performing the action.
Evidence	Evidence can be shown using the invalid index JOptionPane screenshot in the testing section.

9. Conclusion and Reflection

Milestone 2 helped improve the coursework from basic classes into a more complete Java application with a graphical user interface. I used abstraction by making AIModel an abstract class, and I used inheritance through PersonalPlan and ProPlan.

The project also helped me understand polymorphism because the GUI stores different plan objects inside one ArrayList<AIModel>. The instanceof operator is then used to decide which action should be performed. I also practised input validation, JOptionPane error messages, file export and file load features. The updated token-based logic made the program more realistic because prompts now use calculated token values instead of only reducing a prompt count.

10. References

- CS4001 Programming Coursework Requirements and Milestone 2 brief, London Metropolitan University / Islington College, 2025-2026.
- Milestone 2 lecturer clarification message about abstract AIModel, numeric contextWindow and token-based plan logic.
- Oracle Java Documentation. Java Platform, Standard Edition documentation for Swing, ArrayList, exception handling and file I/O.

11. Appendix

Appendix A: AIModel.java

```
public abstract class AIModel {
    private String modelName;
    private double cost;
    private int parameters;
    private int contextWindow;

    public AIModel(String modelName, double cost, int parameters, int contextWindow) {
        this.modelName = modelName;
        this.cost = cost;
        this.parameters = parameters;
        this.contextWindow = contextWindow;
    }

    public String getModelName() {
        return modelName;
    }

    public double getCost() {
        return cost;
    }

    public int getParameters() {
        return parameters;
    }

    public int getContextWindow() {
        return contextWindow;
    }

    public int calculateTotalToken(String promptText, int expectedOutputTokens) {
        int inputTokens = 0;
    }
}
```

```

        if (promptText != null && !promptText.trim().isEmpty()) {
            String[] words = promptText.trim().split("\\s+");
            inputTokens = words.length;
        }

        int totalTokens = inputTokens + expectedOutputTokens;

        if (totalTokens > contextWindow) {
            throw new IllegalArgumentException("Total tokens exceed context window.");
        }

        return totalTokens;
    }

    public abstract String display();
}

```

Appendix B: PersonalPlan.java

```

public class PersonalPlan extends AIModel {
    private int availableTokens;

    public PersonalPlan(String modelName, double price, int parameters, int contextWindow, int
availableTokens) {
        super(modelName, price, parameters, contextWindow);
        this.availableTokens = availableTokens;
    }

    public int getAvailableTokens() {
        return availableTokens;
    }

    public String purchaseTokens(int tokens) {
        if (tokens < 0) {
            return "Error: Please enter a positive token value or upgrade to Pro Plan.";
        }

        availableTokens += tokens;
        return "Tokens added successfully. Available tokens: " + availableTokens;
    }

    public String enterPrompt(String promptText, int expectedOutputTokens) {
        try {
            int totalTokens = calculateTotalToken(promptText, expectedOutputTokens);

            if (totalTokens <= availableTokens) {
                availableTokens -= totalTokens;
                return "Prompt: " + promptText +
                    "\nExpected output tokens: " + expectedOutputTokens +
                    "\nTotal tokens used: " + totalTokens +
                    "\nRemaining tokens: " + availableTokens;
            } else {
                return "Not enough available tokens in Personal Plan.";
            }
        } catch (IllegalArgumentException e) {
            return "Error: " + e.getMessage();
        }
    }
}

```

```

@Override
public String display() {
    return "AI Model: " + getModelName() +
        "\nSubscription Cost: Rs. " + getCost() + " per 1 Lakh tokens" +
        "\nModel Size: " + getParameters() + " billion params" +
        "\nContext Window: " + getContextWindow() +
        "\nAvailable Tokens: " + availableTokens;
}
}

```

Appendix C: ProPlan.java

```

public class ProPlan extends AIModel {
    private int teamSlots;

    public ProPlan(String modelName, double price, int parameters, int contextWindow, int
teamSlots) {
        super(modelName, price, parameters, contextWindow);
        this.teamSlots = teamSlots;
    }

    public int getTeamSlots() {
        return teamSlots;
    }

    public String addTeamMember(String memberName) {
        if (teamSlots > 0) {
            teamSlots--;
            return memberName + " added. Slots remaining: " + teamSlots;
        }
        return "Error: No available team slots.";
    }

    public String removeTeamMember(String memberName) {
        teamSlots++;
        return memberName + " removed. Slots available: " + teamSlots;
    }

    public String enterPrompt(String promptText, int expectedOutputTokens) {
        try {
            int totalTokens = calculateTotalToken(promptText, expectedOutputTokens);

            return "Prompt: " + promptText +
                "\nExpected output tokens: " + expectedOutputTokens +
                "\nTotal tokens used: " + totalTokens +
                "\nPro Plan token balance is not reduced.";
        } catch (IllegalArgumentException e) {
            return "Error: " + e.getMessage();
        }
    }

    @Override
    public String display() {
        return "AI Model: " + getModelName() +
            "\nSubscription Cost: Rs. " + getCost() + " per 1 Lakh tokens" +
            "\nModel Size: " + getParameters() + " billion params" +
            "\nContext Window: " + getContextWindow() +

```

```

        "\nTeam Slots Available: " + teamSlots;
    }
}

```

Appendix D: SubscriptionGUI.java

```

import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;
import java.awt.geom.*;
import java.util.ArrayList;
import java.io.*;

public class SubscriptionGUI extends JFrame implements ActionListener {

    ArrayList<AIModel> planList = new ArrayList<AIModel>();

    static final Color BG_DARK      = new Color(10, 12, 24);
    static final Color BG_CARD      = new Color(18, 22, 42);
    static final Color ACCENT       = new Color(99, 179, 237);
    static final Color ACCENT2      = new Color(154, 117, 234);
    static final Color SUCCESS      = new Color(72, 199, 142);
    static final Color DANGER       = new Color(252, 100, 94);
    static final Color TEXT_MAIN    = new Color(225, 230, 255);
    static final Color TEXT_DIM     = new Color(120, 130, 165);
    static final Color BORDER_CLR   = new Color(40, 50, 90);

    JTextField tfName, tfPrice, tfParams, tfContext,
                tfQuotaSlots, tfPromptText, tfResponseLen,
                tfMemberName, tfIndex;

    JTextArea outputArea;

    JButton btnAddPersonal, btnAddPro, btnDisplayAll, btnClear,
            btnGivePrompt, btnAddMember, btnRemoveMember,
            btnCheckType, btnExport, btnLoad;

    public SubscriptionGUI() {
        setTitle("AI Subscription Manager");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(900, 680);
        setLocationRelativeTo(null);
        setBackground(BG_DARK);

        JPanel root = new JPanel(new BorderLayout(0, 0));
        root.setBackground(BG_DARK);
        root.setBorder(new EmptyBorder(16, 16, 16, 16));
        setContentPane(root);

        root.add(buildHeader(), BorderLayout.NORTH);
        root.add(buildCenter(), BorderLayout.CENTER);
        root.add(buildOutputPanel(), BorderLayout.SOUTH);
    }

    private JPanel buildHeader() {
        JPanel p = new JPanel(new BorderLayout());
        p.setBackground(BG_DARK);
    }
}

```

CS4001NI Programming - Milestone 2 Report

```
p.setBorder(new EmptyBorder(0, 0, 14, 0));

JLabel title = new JLabel("AI Model Subscription Manager");
title.setFont(new Font("Monospaced", Font.BOLD, 20));
title.setForeground(ACCENT);

JLabel sub = new JLabel("CS4001 Java OOP Coursework");
sub.setFont(new Font("Monospaced", Font.PLAIN, 11));
sub.setForeground(TEXT_DIM);

JPanel left = new JPanel(new GridLayout(2, 1, 2, 2));
left.setBackground(BG_DARK);
left.add(title);
left.add(sub);
p.add(left, BorderLayout.WEST);

return p;
}

private JPanel buildCenter() {
    JPanel center = new JPanel(new GridLayout(1, 2, 12, 0));
    center.setBackground(BG_DARK);
    center.add(buildInputCard());
    center.add(buildActionsCard());
    return center;
}

private JPanel buildInputCard() {
    JPanel card = createCard("Plan Details");

    tfName      = addRow(card, "Model Name", "e.g. DeepSeek-R1");
    tfPrice     = addRow(card, "Price (Rs/1L tok)", "e.g. 350.0");
    tfParams    = addRow(card, "Parameters (B)", "e.g. 90");
    tfContext   = addRow(card, "Context Window", "e.g. 96000");
    tfQuotaSlots = addRow(card, "Tokens / Slots", "Personal=Tokens, Pro=Slots");

    card.add(Box.createVerticalStrut(10));
    addSectionLabel(card, "Personal / Pro Prompt");
    tfPromptText = addRow(card, "Prompt Text", "Enter your query...");
    tfResponseLen = addRow(card, "Output Tokens", "e.g. 300");

    card.add(Box.createVerticalStrut(6));
    addSectionLabel(card, "Pro Plan Actions");
    tfMemberName = addRow(card, "Member Name", "e.g. Sita Sharma");

    card.add(Box.createVerticalStrut(6));
    addSectionLabel(card, "Index Lookup");
    tfIndex      = addRow(card, "Index Number", "0, 1, 2 ...");

    card.add(Box.createVerticalGlue());
    return card;
}

private JPanel buildActionsCard() {
    JPanel card = createCard("Actions");

    addSectionLabel(card, "ADD PLANS");
```

CS4001NI Programming - Milestone 2 Report

```
btnAddPersonal = addBtn(card, "Add Personal Plan", ACCENT);
btnAddPro      = addBtn(card, "Add Pro Plan", ACCENT2);

card.add(Box.createVerticalStrut(8));
addSectionLabel(card, "VIEW & MANAGE");
btnDisplayAll  = addBtn(card, "Display All Plans", new Color(60, 80, 140));
btnCheckType   = addBtn(card, "Check Plan Type", new Color(60, 80, 140));
btnClear       = addBtn(card, "Clear Fields", new Color(50, 55, 80));

card.add(Box.createVerticalStrut(8));
addSectionLabel(card, "PROMPT");
btnGivePrompt  = addBtn(card, "Give a Prompt", SUCCESS);

card.add(Box.createVerticalStrut(8));
addSectionLabel(card, "PRO PLAN");
btnAddMember   = addBtn(card, "Add Team Member", ACCENT2);
btnRemoveMember = addBtn(card, "Remove Team Member", DANGER);

card.add(Box.createVerticalStrut(8));
addSectionLabel(card, "FILE I/O");
btnExport      = addBtn(card, "Export to File", new Color(60, 100, 80));
btnLoad        = addBtn(card, "Load from File", new Color(60, 100, 80));

card.add(Box.createVerticalGlue());
return card;
}

private JPanel buildOutputPanel() {
    JPanel p = new JPanel(new BorderLayout());
    p.setBackground(BG_DARK);
    p.setBorder(new EmptyBorder(12, 0, 0, 0));

    JLabel lbl = new JLabel("Output Console");
    lbl.setFont(new Font("Monospaced", Font.BOLD, 12));
    lbl.setForeground(ACCENT);
    lbl.setBorder(new EmptyBorder(0, 0, 4, 0));

    outputArea = new JTextArea(8, 40);
    outputArea.setBackground(new Color(6, 8, 18));
    outputArea.setForeground(SUCCESS);
    outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
    outputArea.setCaretColor(ACCENT);
    outputArea.setEditable(false);
    outputArea.setBorder(new EmptyBorder(10, 12, 10, 12));
    outputArea.setText("System ready. Add a plan to begin.\n");

    JScrollPane scroll = new JScrollPane(outputArea);
    scroll.setBorder(BorderFactory.createLineBorder(BORDER_CLR, 1));
    scroll.setBackground(BG_DARK);

    p.add(lbl, BorderLayout.NORTH);
    p.add(scroll, BorderLayout.CENTER);
    return p;
}

public void actionPerformed(ActionEvent e) {
    Object src = e.getSource();
```

```

if (src == btnAddPersonal) {
    try {
        String name = tfName.getText().trim();
        double price = Double.parseDouble(tfPrice.getText().trim());
        int params = Integer.parseInt(tfParams.getText().trim());
        int context = Integer.parseInt(tfContext.getText().trim());
        int tokens = Integer.parseInt(tfQuotaSlots.getText().trim());

        if (name.isEmpty()) {
            throw new Exception("Model name is empty.");
        }

        planList.add(new PersonalPlan(name, price, params, context, tokens));
        print("Personal Plan added at index [" + (planList.size() - 1) + "]");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Invalid input for Personal Plan.", "Input
Error", JOptionPane.ERROR_MESSAGE);
    }

} else if (src == btnAddPro) {
    try {
        String name = tfName.getText().trim();
        double price = Double.parseDouble(tfPrice.getText().trim());
        int params = Integer.parseInt(tfParams.getText().trim());
        int context = Integer.parseInt(tfContext.getText().trim());
        int slots = Integer.parseInt(tfQuotaSlots.getText().trim());

        if (name.isEmpty()) {
            throw new Exception("Model name is empty.");
        }

        planList.add(new ProPlan(name, price, params, context, slots));
        print("Pro Plan added at index [" + (planList.size() - 1) + "]");
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Invalid input for Pro Plan.", "Input
Error", JOptionPane.ERROR_MESSAGE);
    }

} else if (src == btnDisplayAll) {
    if (planList.isEmpty()) {
        print("No plans added yet.");
        return;
    }

    outputArea.setText("");
    for (int i = 0; i < planList.size(); i++) {
        print("----- Plan Index: " + i + " -----");
        print(planList.get(i).display());
        print("");
    }

} else if (src == btnClear) {
    tfName.setText("");
    tfPrice.setText("");
    tfParams.setText("");
    tfContext.setText("");

```



```

tfQuotaSlots.setText("");
tfPromptText.setText("");
tfResponseLen.setText("");
tfMemberName.setText("");
tfIndex.setText("");
print("Fields cleared.");

} else if (src == btnGivePrompt) {
    int idx = getIndex();
    if (idx == -1) {
        return;
    }

    try {
        String promptText = tfPromptText.getText().trim();
        int responseLen = Integer.parseInt(tfResponseLen.getText().trim());

        AIModel obj = planList.get(idx);

        if (obj instanceof PersonalPlan) {
            PersonalPlan pp = (PersonalPlan) obj;
            print(pp.enterPrompt(promptText, responseLen));
        } else if (obj instanceof ProPlan) {
            ProPlan pro = (ProPlan) obj;
            print(pro.enterPrompt(promptText, responseLen));
        } else {
            print("Unknown plan type.");
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Please enter valid output tokens.", "Input
Error", JOptionPane.ERROR_MESSAGE);
    }

} else if (src == btnAddMember) {
    int idx = getIndex();
    if (idx == -1) {
        return;
    }

    AIModel obj = planList.get(idx);
    if (obj instanceof ProPlan) {
        ProPlan pro = (ProPlan) obj;
        print(pro.addTeamMember(tfMemberName.getText().trim()));
    } else {
        JOptionPane.showMessageDialog(this, "This operation is only for Pro Plan.",
"Type Error", JOptionPane.ERROR_MESSAGE);
    }

} else if (src == btnRemoveMember) {
    int idx = getIndex();
    if (idx == -1) {
        return;
    }

    AIModel obj = planList.get(idx);
    if (obj instanceof ProPlan) {
        ProPlan pro = (ProPlan) obj;
        print(pro.removeTeamMember(tfMemberName.getText().trim()));
    }

```

CS4001NI Programming - Milestone 2 Report

```

        } else {
            JOptionPane.showMessageDialog(this, "This operation is only for Pro Plan.",
                "Type Error", JOptionPane.ERROR_MESSAGE);
        }

        } else if (src == btnCheckType) {
            int idx = getIndex();
            if (idx == -1) {
                return;
            }

            AIModel obj = planList.get(idx);

            if (obj instanceof PersonalPlan) {
                print("Plan at index [" + idx + "] is Personal Plan.");
            } else if (obj instanceof ProPlan) {
                print("Plan at index [" + idx + "] is Pro Plan.");
            } else {
                print("Unknown plan type.");
            }

        } else if (src == btnExport) {
            exportToFile();

        } else if (src == btnLoad) {
            loadFromFile();
        }
    }

    private int getIndex() {
        int idx = -1;

        try {
            idx = Integer.parseInt(tfIndex.getText().trim());

            if (idx < 0 || idx >= planList.size()) {
                JOptionPane.showMessageDialog(this,
                    "Index out of range. Valid index is from 0 to " + (planList.size() -
1),
                    "Index Error",
                    JOptionPane.ERROR_MESSAGE);
                return -1;
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this,
                "Please enter a valid integer index.",
                "Input Error",
                JOptionPane.ERROR_MESSAGE);
            return -1;
        }

        return idx;
    }

    private void exportToFile() {
        try {
            PrintWriter pw = new PrintWriter(new FileWriter("plans_export.txt"));

```

```

        for (int i = 0; i < planList.size(); i++) {
            pw.println("=== Plan [" + i + "] ===");
            pw.println(planList.get(i).display());
            pw.println();
        }

        pw.close();
        print("Exported successfully to plans_export.txt");
    } catch (IOException ex) {
        print("Export failed: " + ex.getMessage());
    }
}

private void loadFromFile() {
    try {
        BufferedReader br = new BufferedReader(new FileReader("plans_export.txt"));
        outputArea.setText("");

        String line;
        while ((line = br.readLine()) != null) {
            outputArea.append(line + "\n");
        }

        br.close();
        print("File loaded successfully.");
    } catch (IOException ex) {
        print("Load failed: " + ex.getMessage());
    }
}

private void print(String msg) {
    outputArea.append(msg + "\n");
    outputArea.setCaretPosition(outputArea.getDocument().getLength());
}

private JPanel createCard(String title) {
    JPanel card = new JPanel();
    card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
    card.setBackground(BG_CARD);
    card.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(BORDER_CLR, 1),
        new EmptyBorder(14, 16, 14, 16)
    ));

    JLabel lbl = new JLabel(title);
    lbl.setFont(new Font("Monospaced", Font.BOLD, 13));
    lbl.setForeground(ACCENT);
    lbl.setAlignmentX(LEFT_ALIGNMENT);
    lbl.setBorder(new EmptyBorder(0, 0, 10, 0));
    card.add(lbl);
    return card;
}

private JTextField addRow(JPanel card, String label, String hint) {
    JPanel row = new JPanel(new BorderLayout(8, 0));
    row.setBackground(BG_CARD);

```

CS4001NI Programming - Milestone 2 Report

```
row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 32));
row.setAlignmentX(LEFT_ALIGNMENT);

JLabel lbl = new JLabel(label);
lbl.setFont(new Font("Monospaced", Font.PLAIN, 11));
lbl.setForeground(TEXT_DIM);
lbl.setPreferredSize(new Dimension(148, 24));

JTextField tf = new JTextField();
tf.setFont(new Font("Monospaced", Font.PLAIN, 12));
tf.setBackground(new Color(14, 18, 36));
tf.setForeground(TEXT_MAIN);
tf.setCaretColor(ACCENT);
tf.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createLineBorder(BORDER_CLR, 1),
    new EmptyBorder(2, 6, 2, 6)
));
tf.setToolTipText(hint);

row.add(lbl, BorderLayout.WEST);
row.add(tf, BorderLayout.CENTER);
card.add(row);
card.add(Box.createVerticalStrut(5));
return tf;
}

private JButton addBtn(JPanel card, String text, Color color) {
    JButton btn = new JButton(text) {
        protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            if (getModel().isPressed()) {
                g2.setColor(color.darker());
            } else if (getModel().isRollover()) {
                g2.setColor(color.brighter());
            } else {
                g2.setColor(new Color(color.getRed(), color.getGreen(), color.getBlue(),
40));
            }

            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 8, 8);
            g2.setColor(color);
            g2.setStroke(new BasicStroke(1.2f));
            g2.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, 8, 8);
            g2.dispose();

            super.paintComponent(g);
        }
    };

    btn.setFont(new Font("Monospaced", Font.BOLD, 12));
    btn.setForeground(color);
    btn.setBackground(new Color(0, 0, 0, 0));
    btn.setOpaque(false);
    btn.setContentAreaFilled(false);
    btn.setBorderPainted(false);
```

CS4001NI Programming - Milestone 2 Report

```
        btn.setFocusPainted(false);
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        btn.setMaximumSize(new Dimension(Integer.MAX_VALUE, 34));
        btn.setAlignmentX(LEFT_ALIGNMENT);
        btn.addActionListener(this);

        card.add(btn);
        card.add(Box.createVerticalStrut(4));
        return btn;
    }

    private void addSectionLabel(JPanel card, String text) {
        JLabel lbl = new JLabel(text);
        lbl.setFont(new Font("Monospaced", Font.BOLD, 10));
        lbl.setForeground(TEXT_DIM);
        lbl.setAlignmentX(LEFT_ALIGNMENT);
        lbl.setBorder(new EmptyBorder(4, 0, 4, 0));
        card.add(lbl);
    }

    public static void main(String[] args) {
        try {
            UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
        } catch (Exception e) {
        }

        SwingUtilities.invokeLater(new Runnable() {
            public void run() {
                new SubscriptionGUI().setVisible(true);
            }
        });
    }
}
```